

Introduction to Machine Learning

Basics, fundamentals, history and context

Prof. Didi Ramsaran Chin

INFO-02020

September 2026

What is Machine Learning (ML)?

[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

—Arthur Samuel, 1959

A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

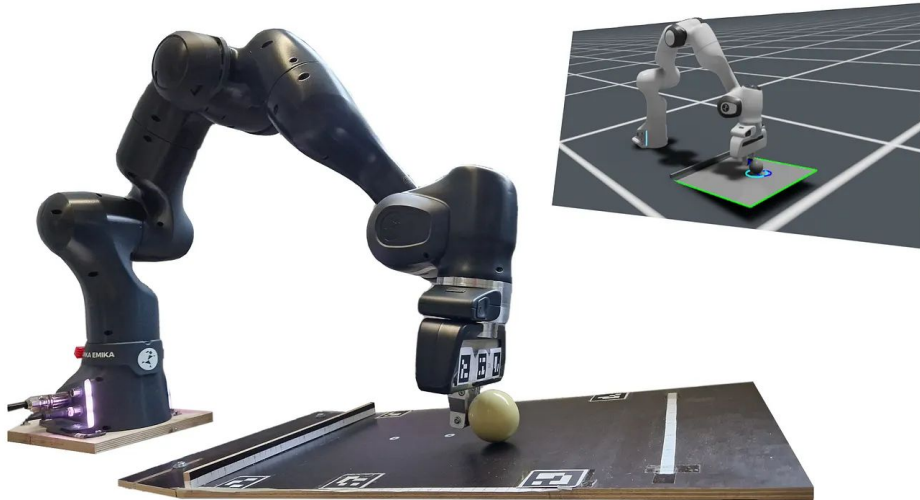
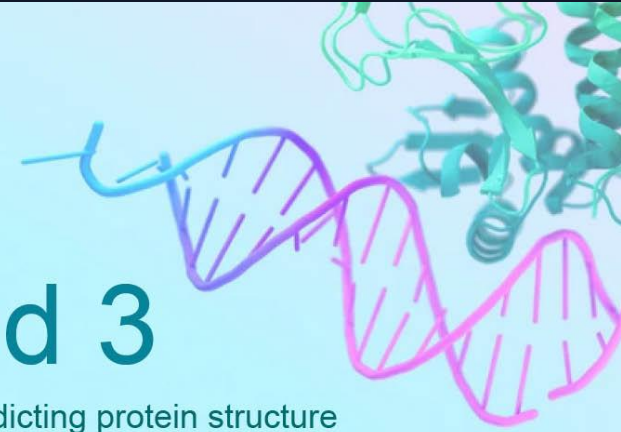
—Tom Mitchell, 1997

Examples of ML applications

Alphafold (Google Deepmind)

AlphaFold 3

A game-changing tool for predicting protein structure



Robotics (ML and control theory)

Source: [From Simulation to Reality: A Practical Journey of Reinforcement Learning in Robotics](#)

Examples of ML applications

Automatic fruit classifier



Robotics (ML and control theory)



Examples of ML applications

Other examples?

More on Machine Learning?

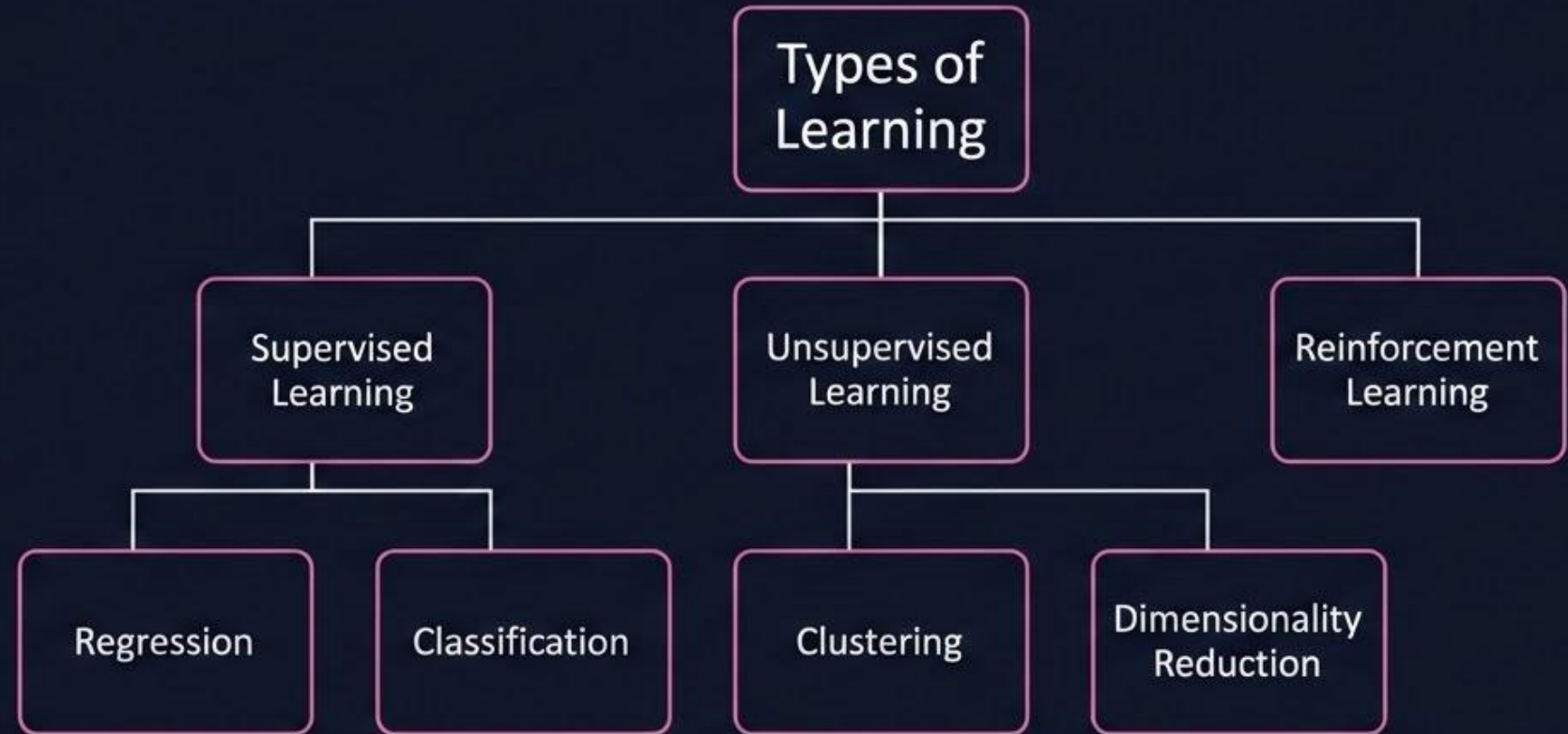
Machine Learning (ML)

encompasses an umbrella of algorithms that are commonly and grossly grouped into three.

Supervised = tags and data

Unsupervised = no tags, just data

RL = No tags and no data, just rules for the agent to follow and optimized its behaviour (aka. developing a policy)



Inspo: [An Overview of Supervised, Unsupervised, and Reinforcement Learning in Machine Learning](#)

More on Machine Learning?

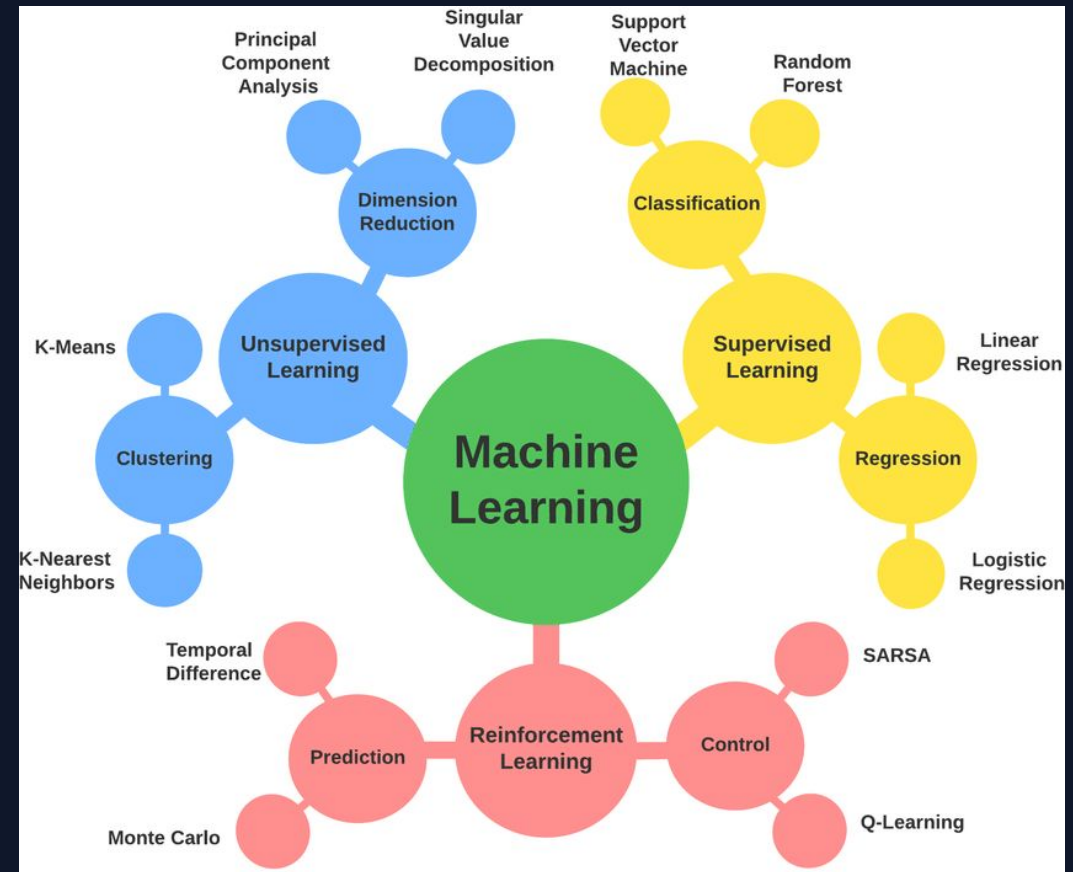
Machine Learning (ML)

encompasses an umbrella of algorithms that are commonly and grossly grouped into three.

Supervised = tags and data

Unsupervised = no tags, just data

RL = No tags and no data, just rules for the agent to follow and optimized its behaviour (aka. developing a policy)



Source: [A Survey and Perspective on Artificial Intelligence for Security-Aware Electronic Design Automation](#)

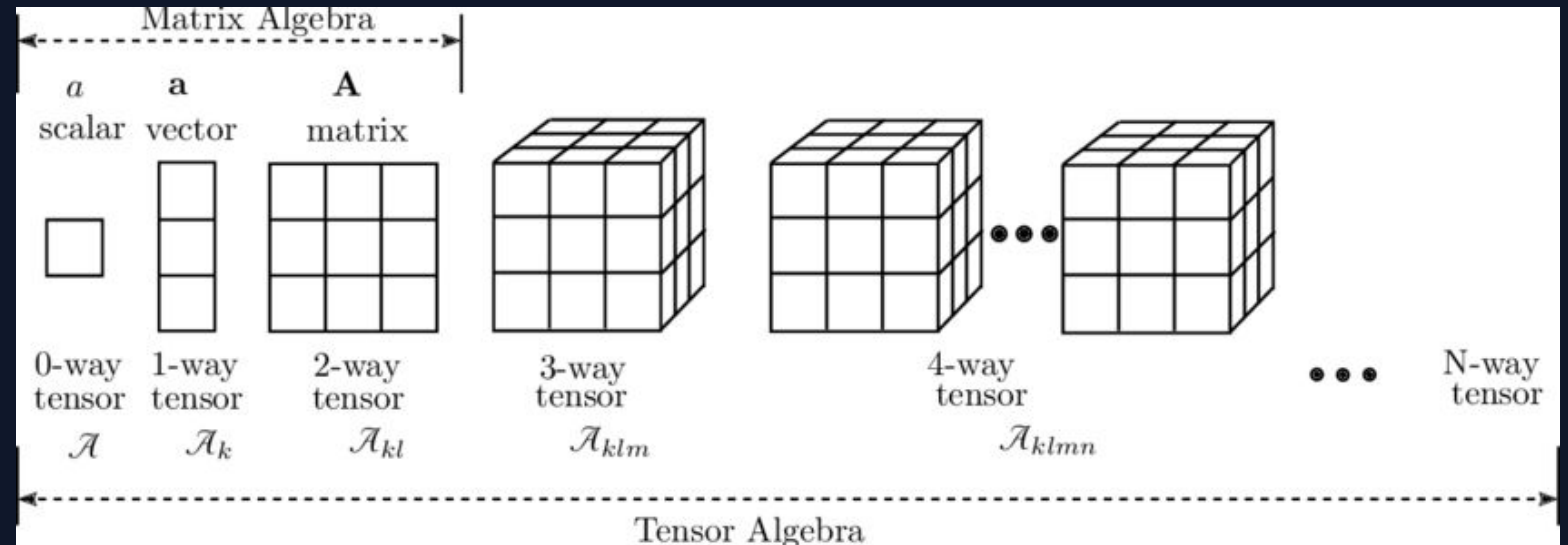
Mathematics of Machine Learning (in general)

Linear Algebra: Vectors and matrices (sums, dot product, matrix mult, etc), and tensors.

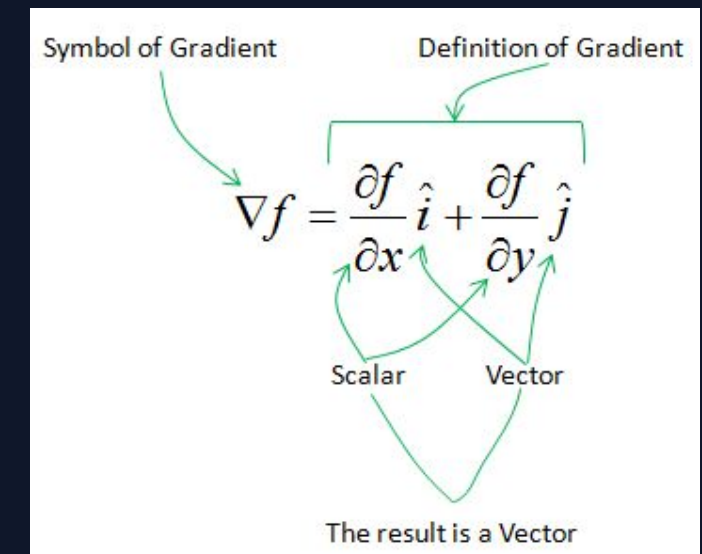
(Vector) Calculus: (Partial) derivatives, gradients (vector of partial derivatives), chain rule.

Probability and Statistics: probs, Random variables, (normal) distributions, Conditional probs, Variance, mathematical expectation, etc.

Other elements: Optimizations (loss/cost functions like MSE, MLE, Cross Entropy), gradient descent, backprop, activations functions (sigmoid, tanh, relu)



Source: <https://static.wikidocs.net/images/page/229560/Tensors.png>



Source: engineering Math | ShareTechnote

Mathematics of Machine Learning (in general)

MSE: n = number of cases, y = real

target, $\hat{y}$ = predicted target

Prediction error for continuous values

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

$$L(\theta) = \prod_{i=1}^N f(x_i; \theta)$$

$$\text{Likelihood} = L(\theta \mid \text{Data}) = P(\text{Data} \mid \theta)$$

MLE: θ = parameter, x = data point

Likelihood of data given a parameter to calculate plausible parameters given data. General data.

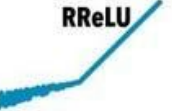
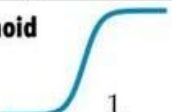
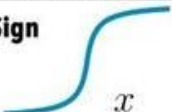
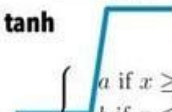
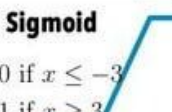
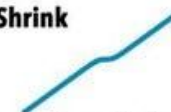
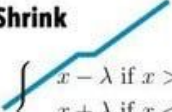
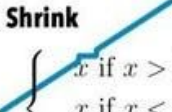
Cross Entropy: y = true label distribution, p = predicted prob. K = categories

differences between predicted and real prob distribution. Categorical data

$$L = - \sum_{k=1}^K y_k \log(p_k)$$

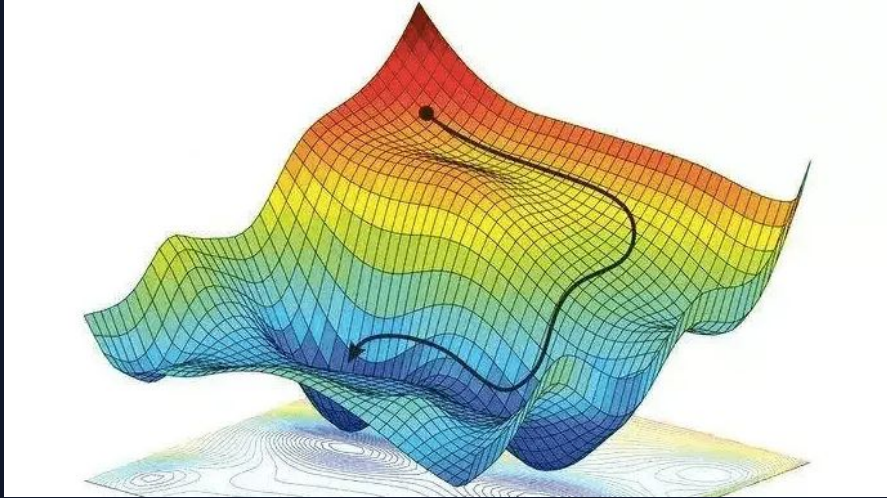
Mathematics of Machine Learning (in general)

Activation functions

Neural Network Activation Functions: a small subset!		
ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

Mathematics of Machine Learning (in general)

Gradient descent



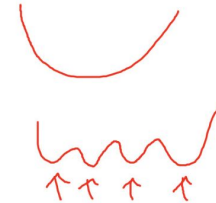
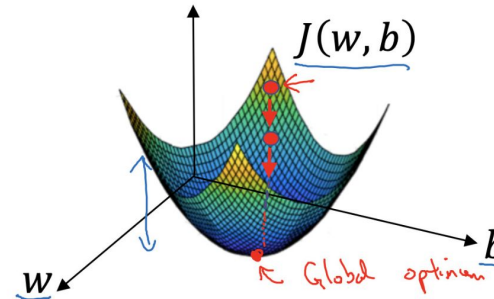
Source:
<https://pub.aimind.so/gradient-descent-and-its-variants-0689c858652b>

Gradient Descent

Recap: $\hat{y} = \sigma(w^T x + b)$, $\sigma(z) = \frac{1}{1+e^{-z}}$ ←

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Want to find w, b that minimize $J(w, b)$



Andrew Ng

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

(for $j = 1$ and $j = 0$)

}

Source:

<https://community.deeplearning.ai/t/gradient-descent-algorithm-computation/382883>

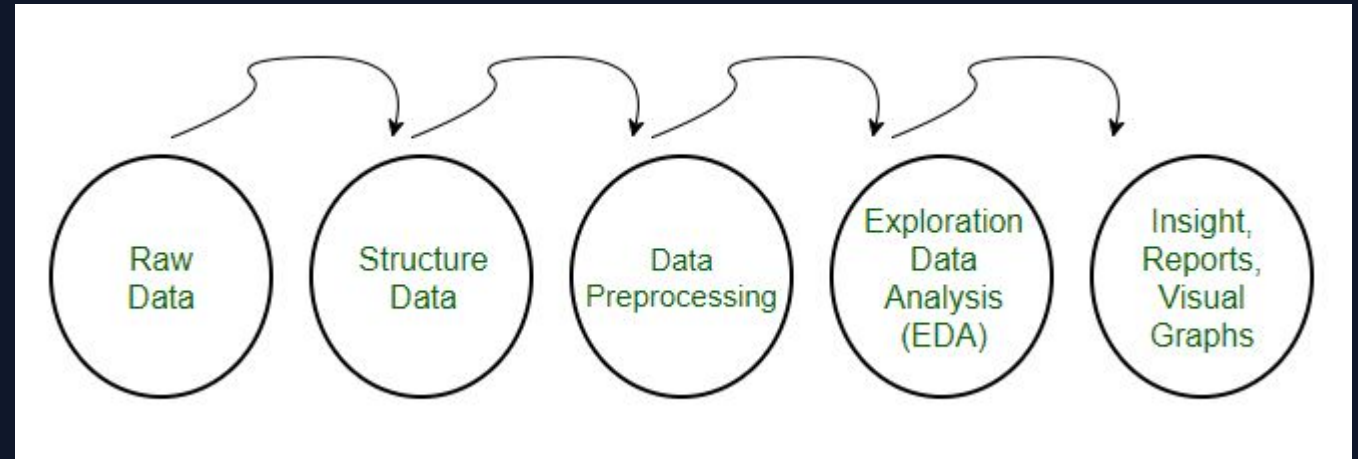
Source:

<https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/>

Challenges of Machine Learning 1

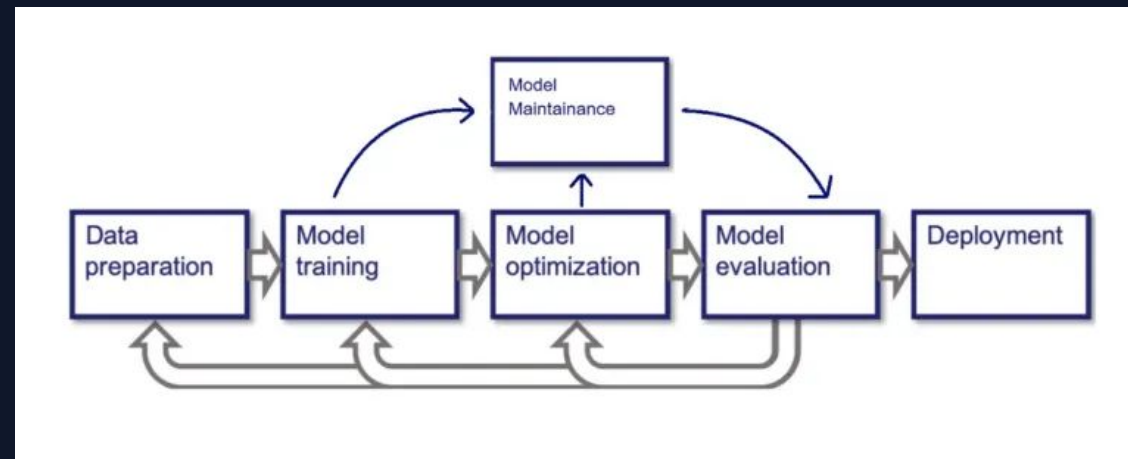
Data handling:

- Is data representative of the phenomena of training? - data gathering and sampling bias.
- Are there any biases?
- Has it been adequately preprocessed?
- Is there enough data?
- Is it good-quality data? - preprocessing
- Are data features relevant? - feature engineering (selection, extraction, new features)



Source:

<https://levelup.gitconnected.com/a-beginners-guide-to-data-preprocessing-in-python-c4c37f632470?gi=08a2d243b0fd>



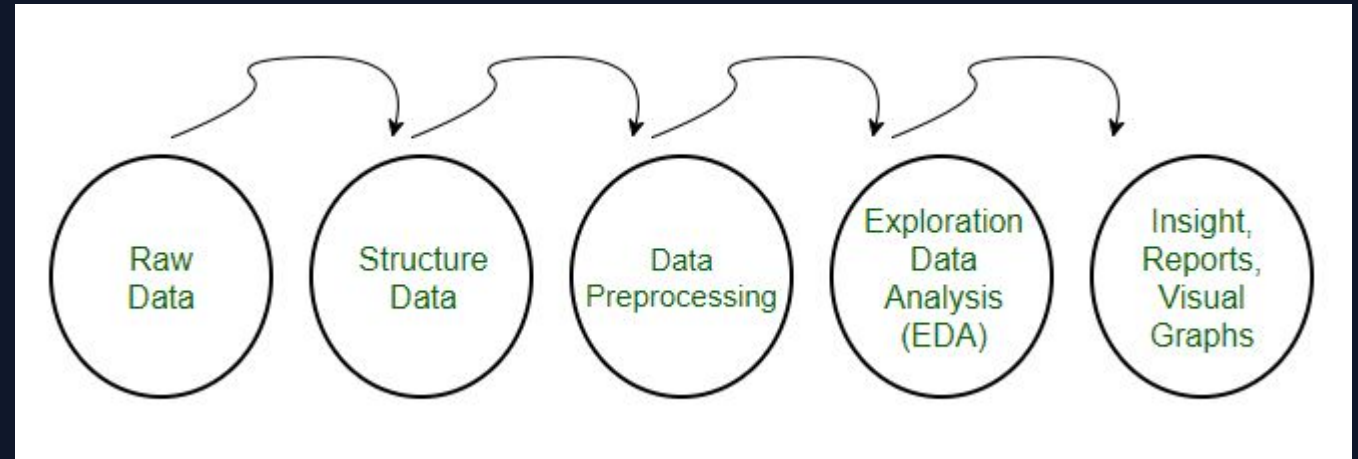
Source: [Boost your pipelines with ColumnTransformer | by Fernandao Lacerda Dantas | Medium](#)

Challenges of Machine Learning 2

Algorithm selection: is the problem we are trying to solve adequately tackled by the algorithm we are using?

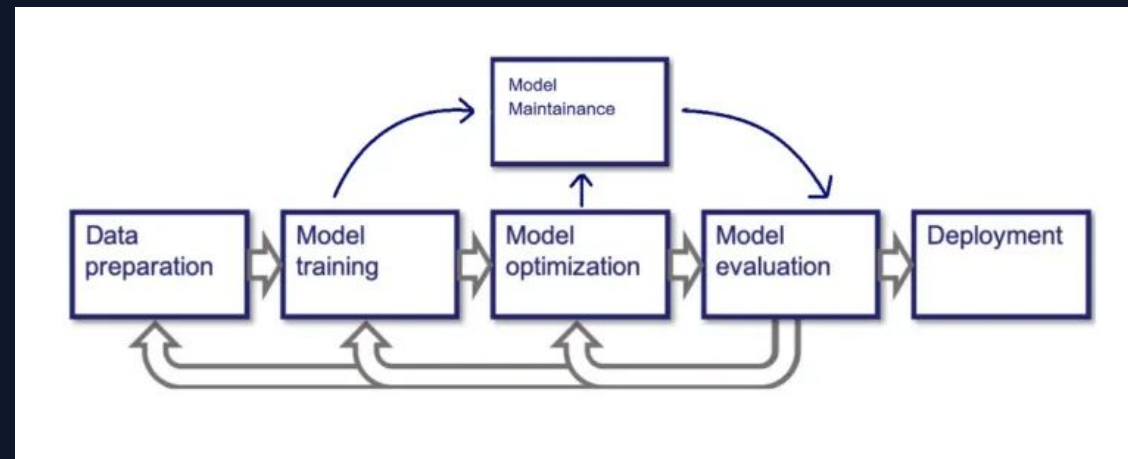
Resources: Is the model efficient (resources vs achievement)? is it too complex?

Code: is the model code parsimonious? Is the code clean and understandable?



Source:

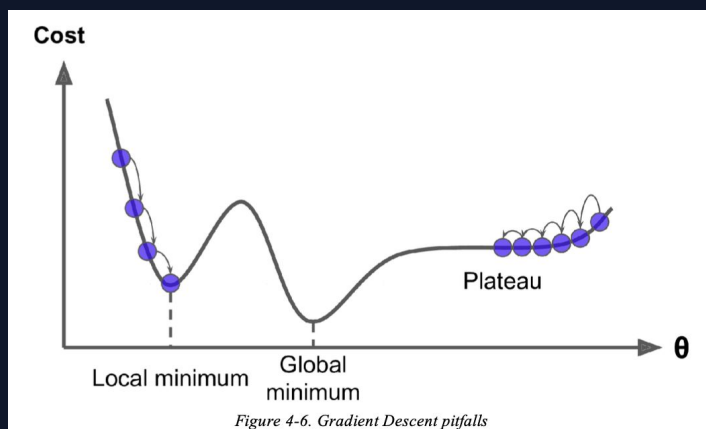
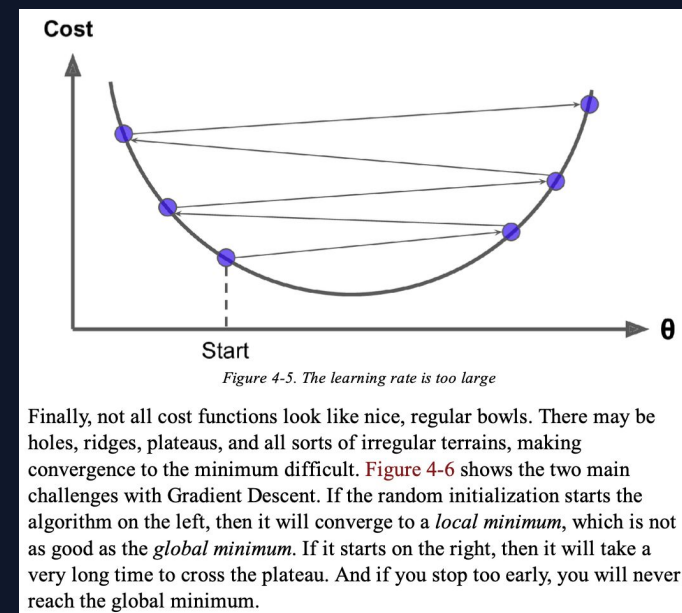
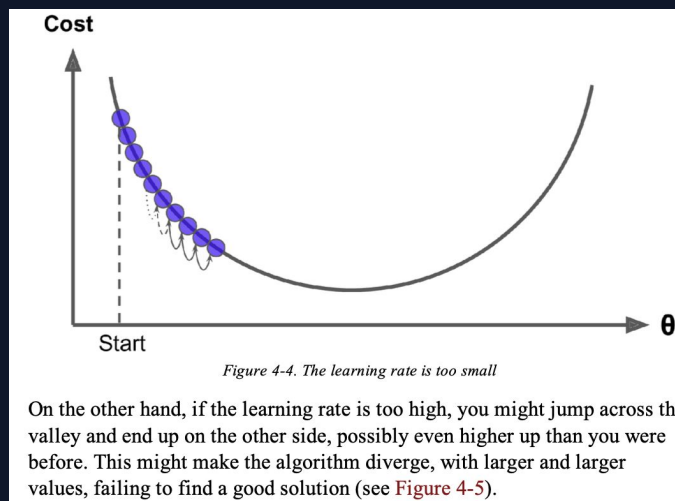
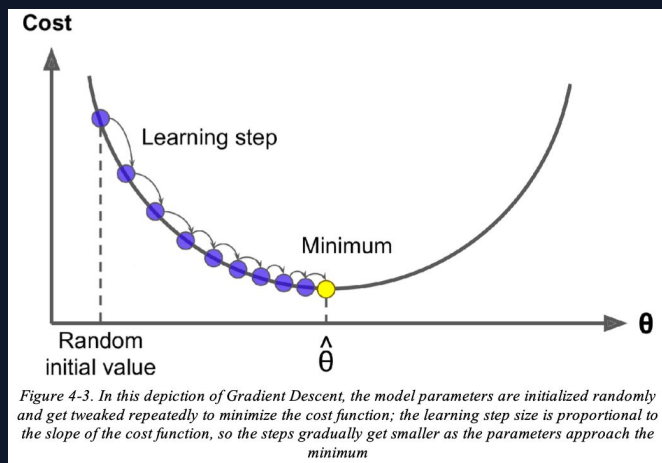
<https://levelup.gitconnected.com/a-beginners-guide-to-data-preprocessing-in-python-c4c37f632470?gi=08a2d243b0fd>



Source: [Boost your pipelines with ColumnTransformer | by Fernandao Lacerda Dantas | Medium](#)

Challenges of Machine Learning 3

Optimization of parameters: Optimizing, step size, plateau. local vs global minima



Challenges of Machine Learning 4

Size of batch for gradient descent:

- **Batch GD:** all training data taking at once, parameters adjustment with GD, repeat E times
- **Mini-batch GD:** randomly select N data points from data, parameters adjustment with GD, repeat T times
- **Stochastic GD:** take each individual data points, parameters adjustment with GD, continue until dataset is covered.

Data	Batch GD	Mini-Batch GD (N=3)	Stochastic GD
x in X: X contains all N in [1, 100]	• Take all N. • Calculate loss. • Adjust with GD. • Repeat E times.	• Take 3 random numbers from N. • Calculate loss. • Adjust with GD. • Repeat T times. • Then repeat E times	• Take one number from N at a time. • Calculate loss. • Adjust with GD. • Grab next number. • complete data set and repeat E times

Challenges of Machine Learning 5

Generalization: Capacity for a model trained on a given data set to extend predictions accurately to unseen data.

Overfitting: over adjusting parameters excessively to training data, copying perfectly pattern from it.

Underfitting: under adjustment of parameters where model is not grasping significant patterns effectively.

Hyperparameter selection: these parameters are not trained on, so we must select it. How? Validation and cross validation will do.

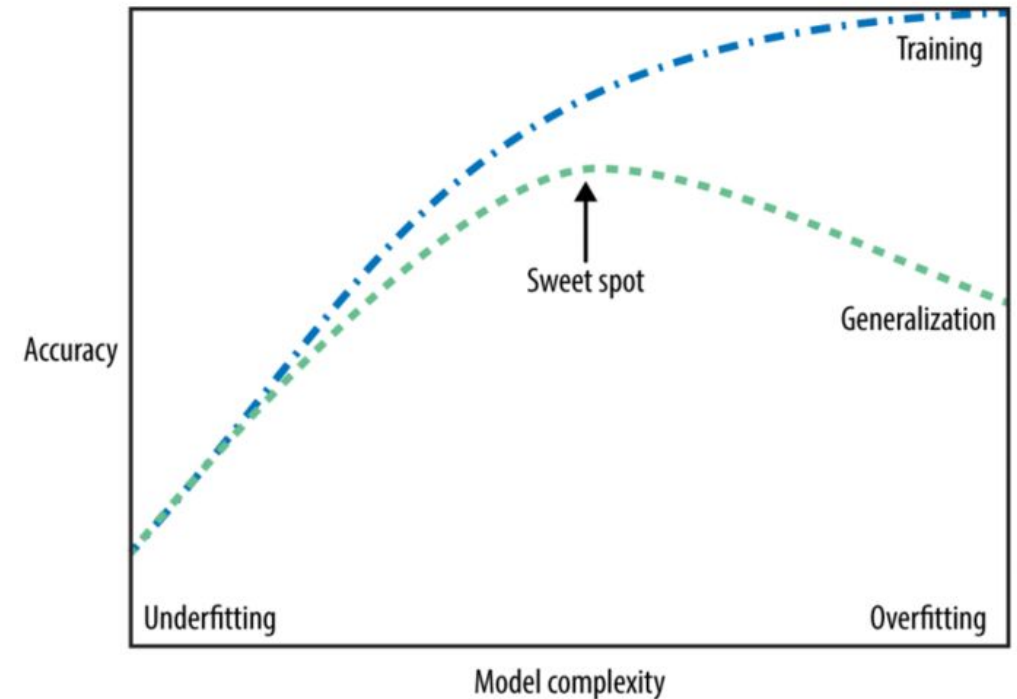


Figure 2-1. Trade-off of model complexity against training and test accuracy

Müller and Guido (2017)

Pipeline of ML development

Preprocessing → Training → Optimization → Testing → Deployment

Another step (within optimization): Validation and Cross-Validation (more computationally costly)

In[3]:

```
mglearn.plots.plot_cross_validation()
```

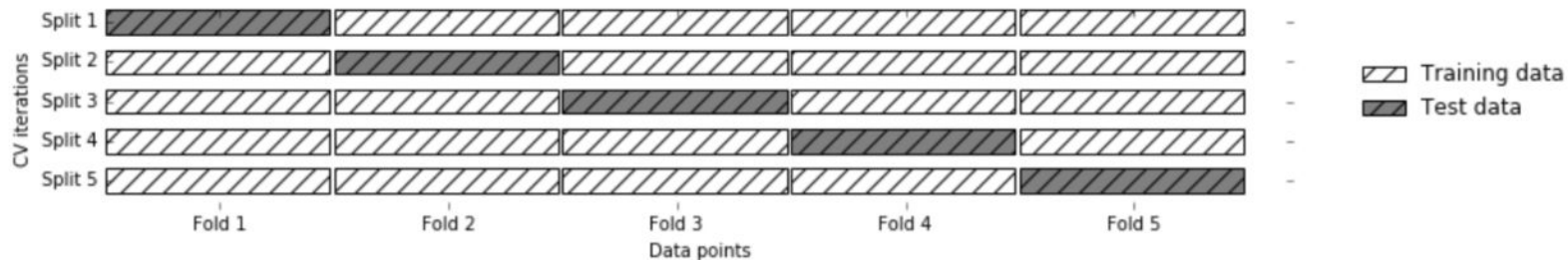
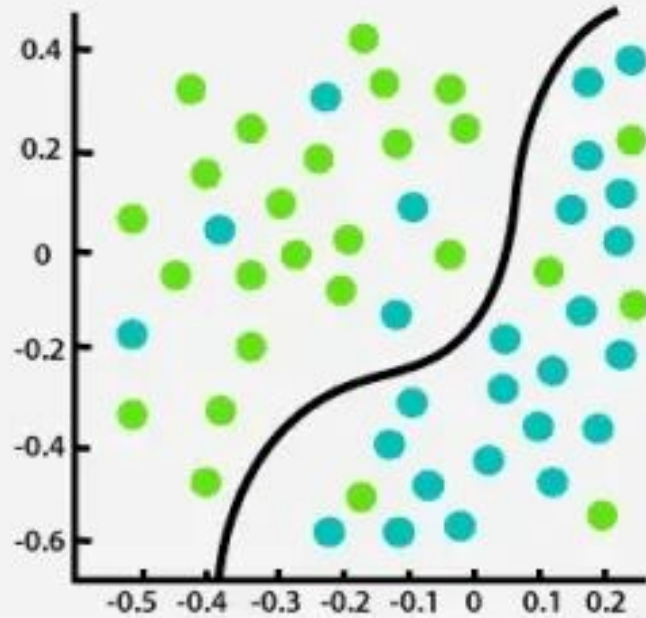
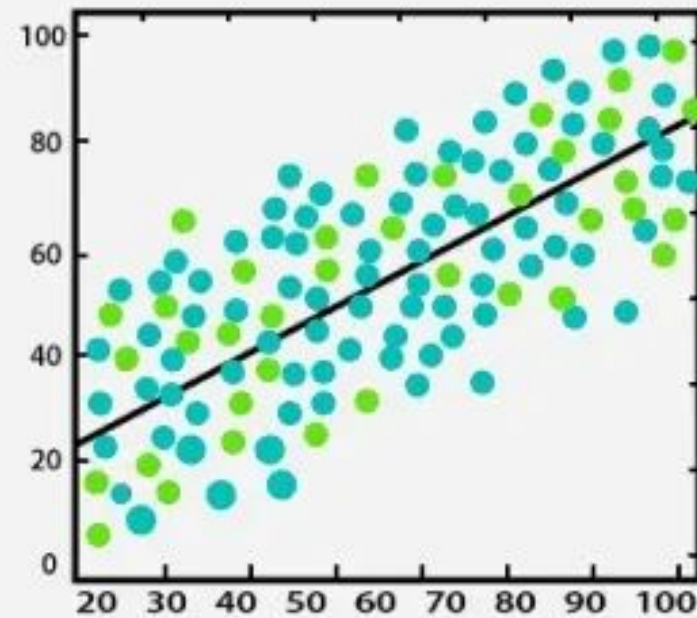


Figure 5-1. Data splitting in five-fold cross-validation

Regression and Classification (Supervised ML)

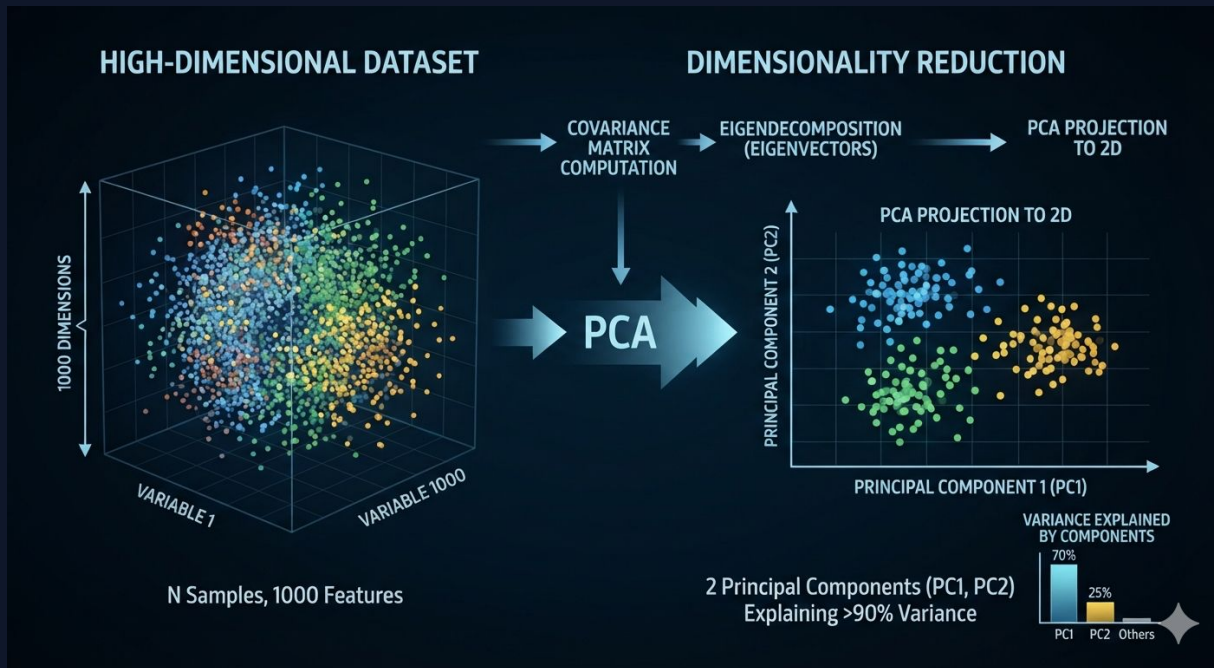


Classification

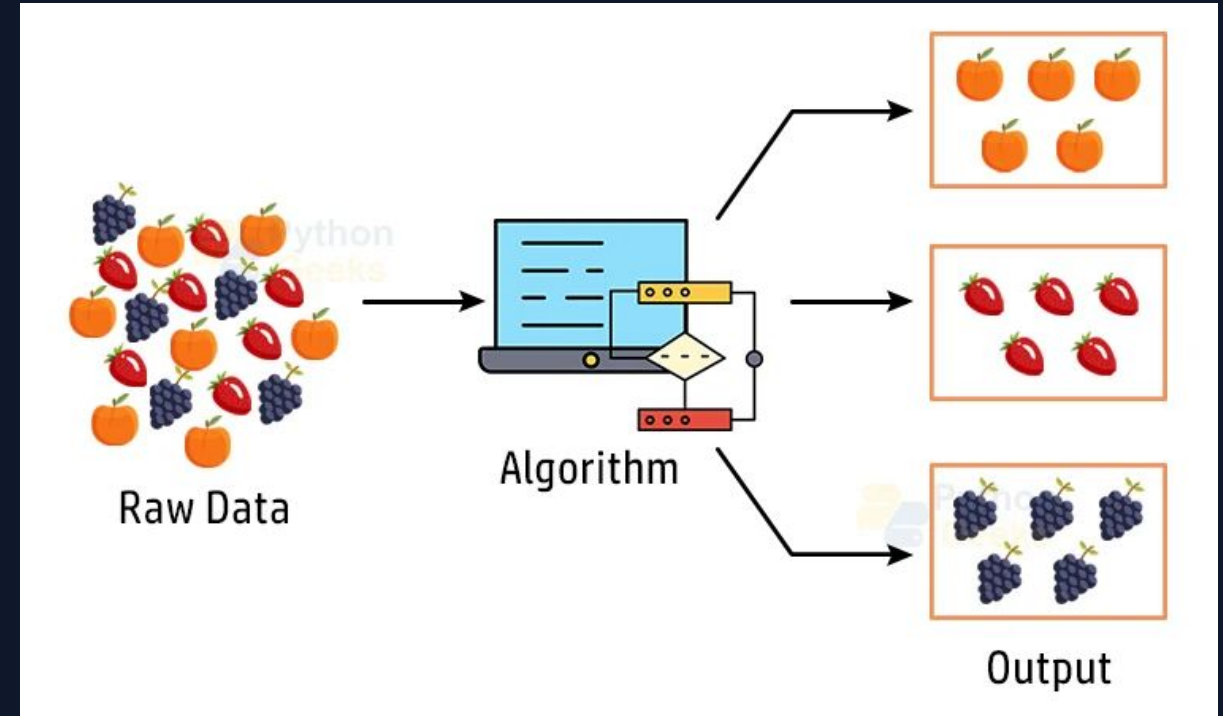


Regression

Regression and Classification (Unsupervised ML)

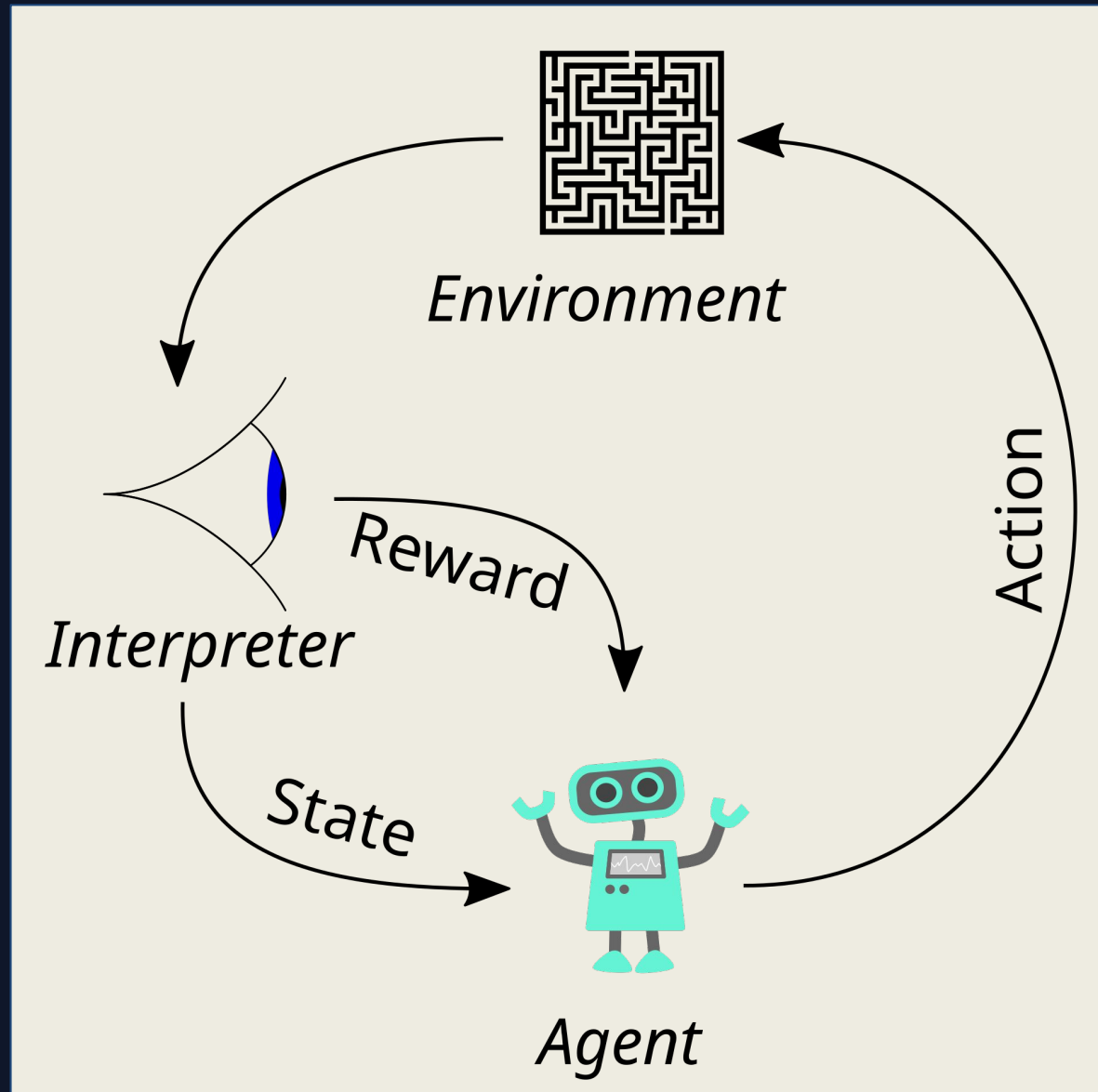


Source: gemini



Source: <https://pythongeeks.org/clustering-in-machine-learning/>

Reinforcement learning



General overview of (almost) all concepts

Data assessment and problem/goals

- Which ML model?
- Data probs:
 - Representativeness
 - Biases
 - Preprocessing
 - Data quantity
 - Data quality
 - features of the data
 - Structure of the data (according to model)

Processing and Exploratory Data Analysis (EDA)

- Preprocessing data
- EDA
 - Statistics and visualization.
 - Mathematical properties.
 - Plotting of data
 - bivariate metrics and correlations
- Data restructuring:
 - Train / Validation / Test sets

Training of models

- Model structure:
 - Inputs
 - Parameters
 - Hyperparameters
 - Outputs
 - Loss calculation and adjustment of parameters

General overview of (almost) all concepts

Training of models

- Regression algorithm
 - Simple and matrix formula
 - x = data point
 - b/β = parameter to optimize
 - $\hat{y}$ = prediction

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

$\square$

$\square$

$$Y = X\beta + \varepsilon$$

General overview of (almost) all concepts

Training of models

- Regression algorithm
 - Simple and matrix formula
 - x = data point
 - b/β = parameter to optimize
 - $\hat{y}$ = prediction
- Activation function: Sigmoid in this case
- Determine loss (e.g. MSE)

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

General overview of (almost) all concepts

Training of models

- Regression algorithm
 - Simple and matrix formula
 - x = data point
 - b/β = parameter to optimize
 - $\hat{y}$ = prediction
- gradient descent to optimize parameters (b/β)
 - Hyperparameter alpha (step size).

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

$$\beta_{next} = \beta_{current} - \alpha \nabla f(\beta_{current})$$

$$\beta_{t+1} = \beta_t - \alpha \nabla J(\beta_t)$$

General overview of (almost) all concepts

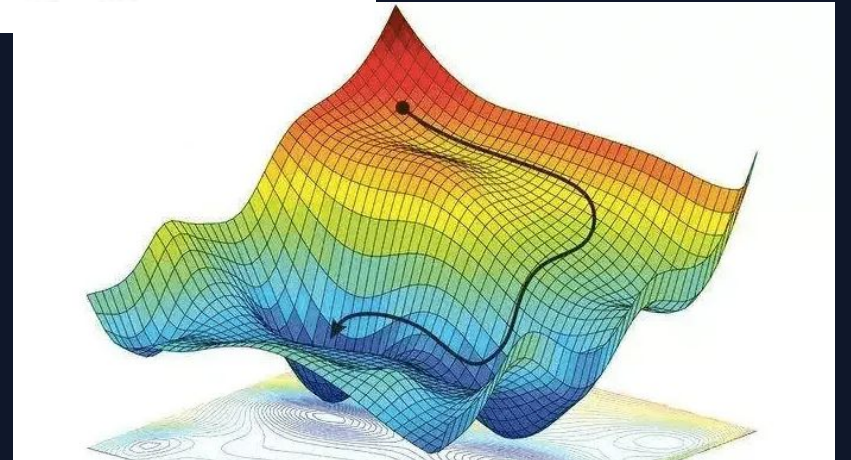
Training of models

- Adjust new parameters
- discard b/beta
- retain b+1/beta+1
- repeat process

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

$$\beta_{next} = \beta_{current} - \alpha \nabla f(\beta_{current})$$

$$\beta_{t+1} = \beta_t - \alpha \nabla J(\beta_t)$$



General overview of (almost) all concepts

Training of models

- After model training
- Select type of model validation: simple or K-fold
- Split accordingly
- Model validation
 - Hyperparameter selection (e.g. alpha).
 - Prevent overfitting, underfitting, plateauing, local minima
 - more robust performance metrics

In[3]:

```
mglearn.plots.plot_cross_validation()
```

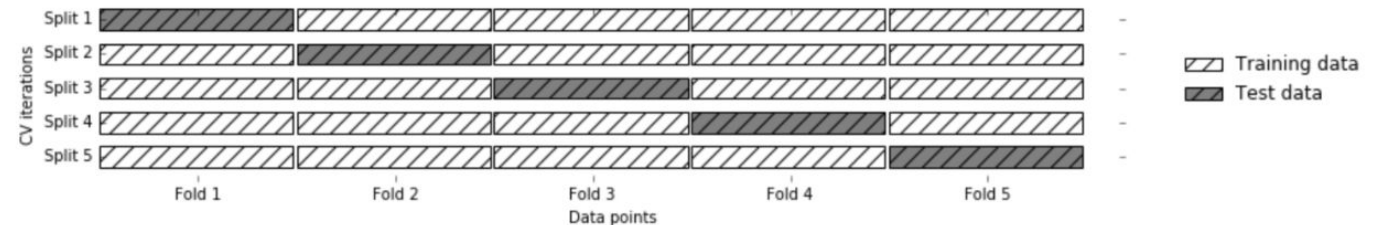


Figure 5-1. Data splitting in five-fold cross-validation

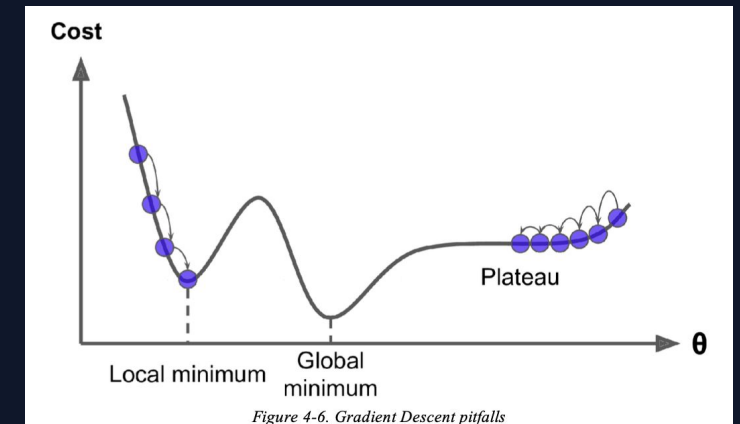


Figure 4-6. Gradient Descent pitfalls

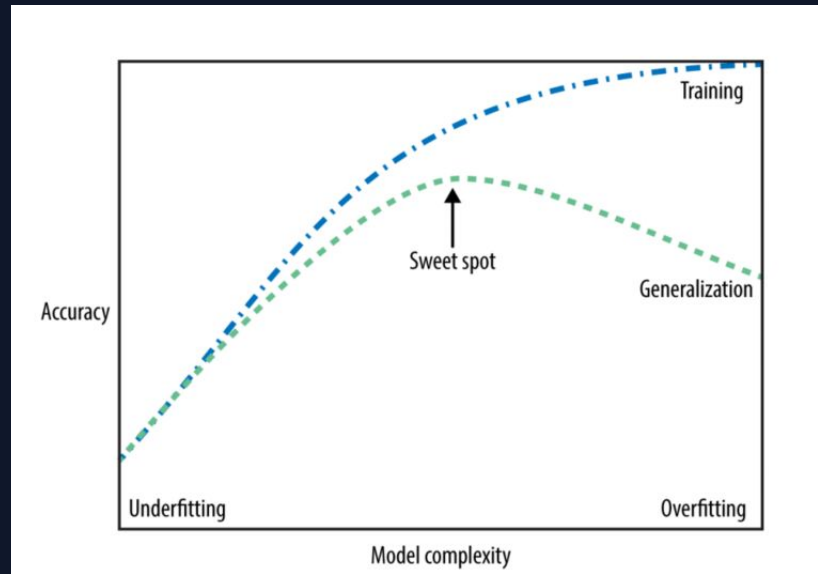
General overview of (almost) all concepts

Training of model

- Model training
- Calculate precision
- Model validation

Testing of model

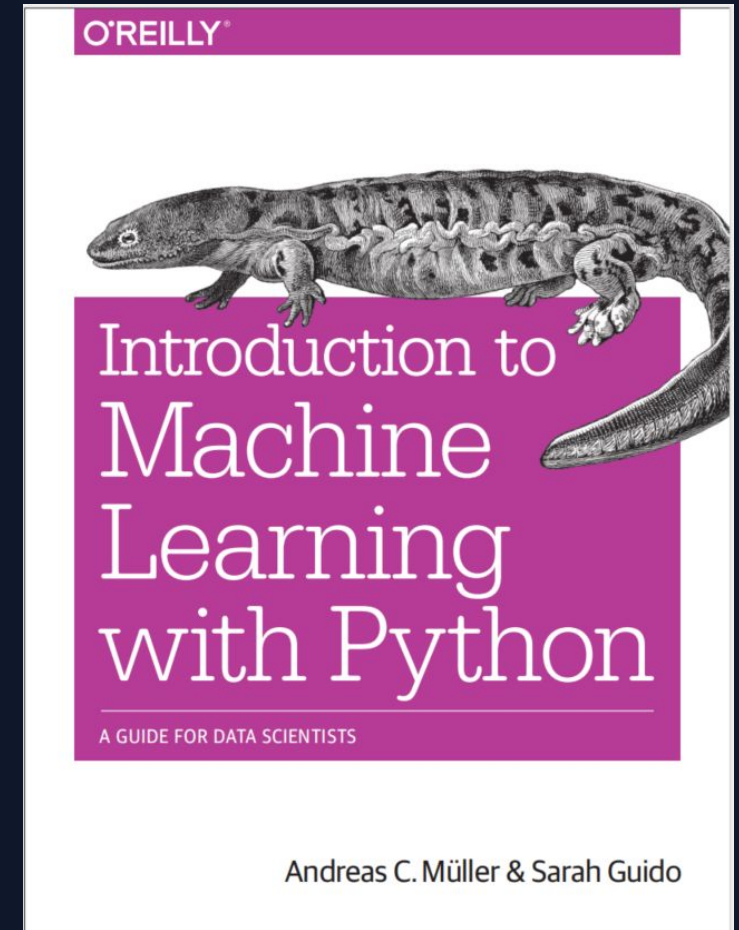
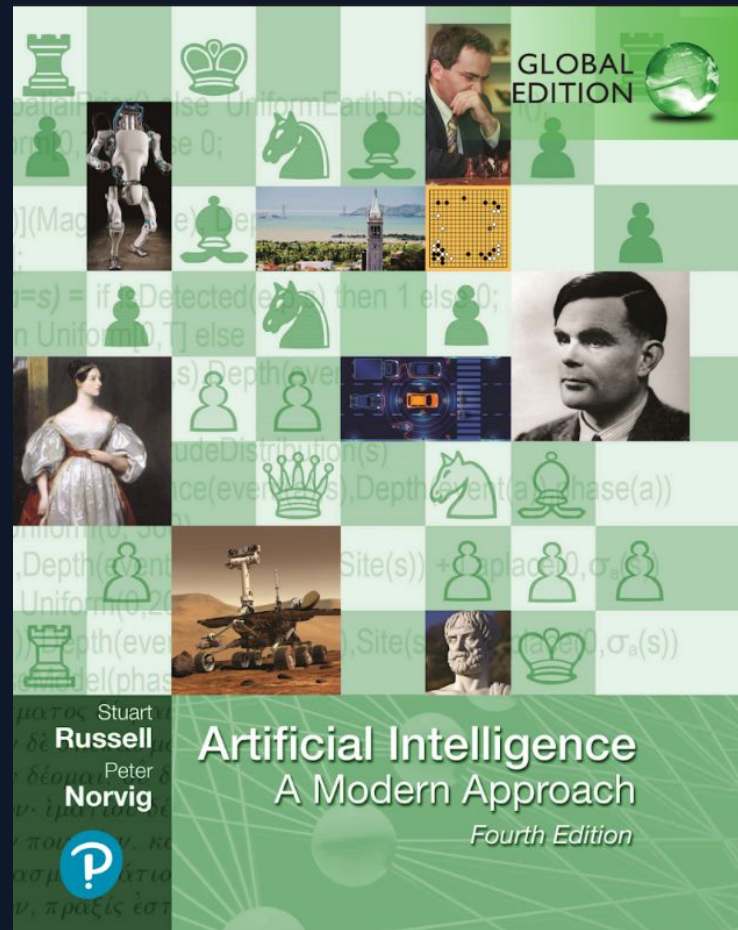
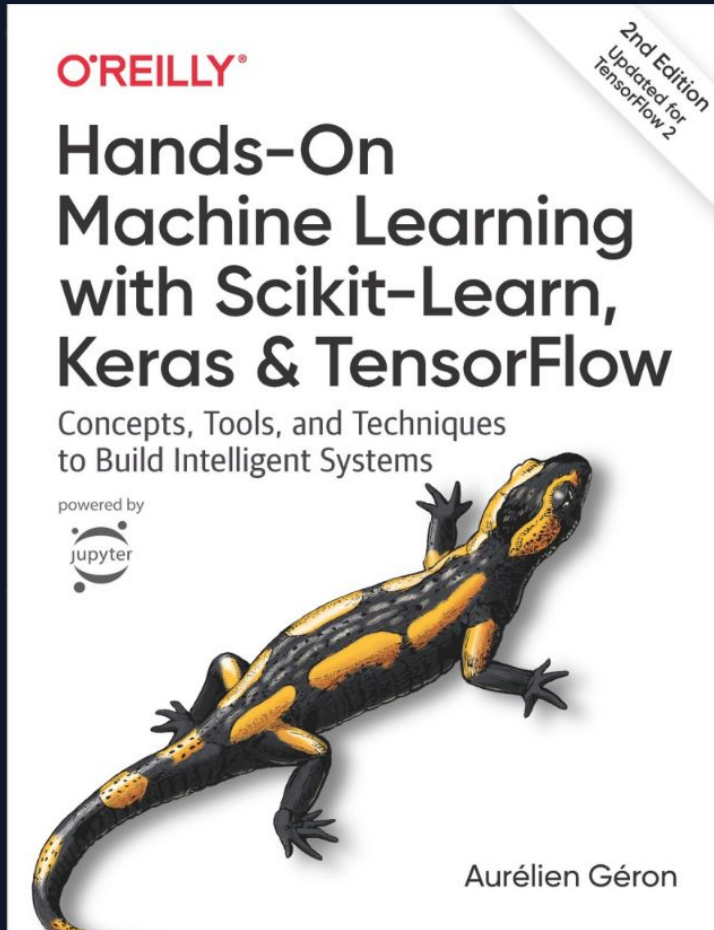
- Make prediction
 - Calculate metric of success (e.g.
 - Evaluate vs accuracy of training at different e in E .



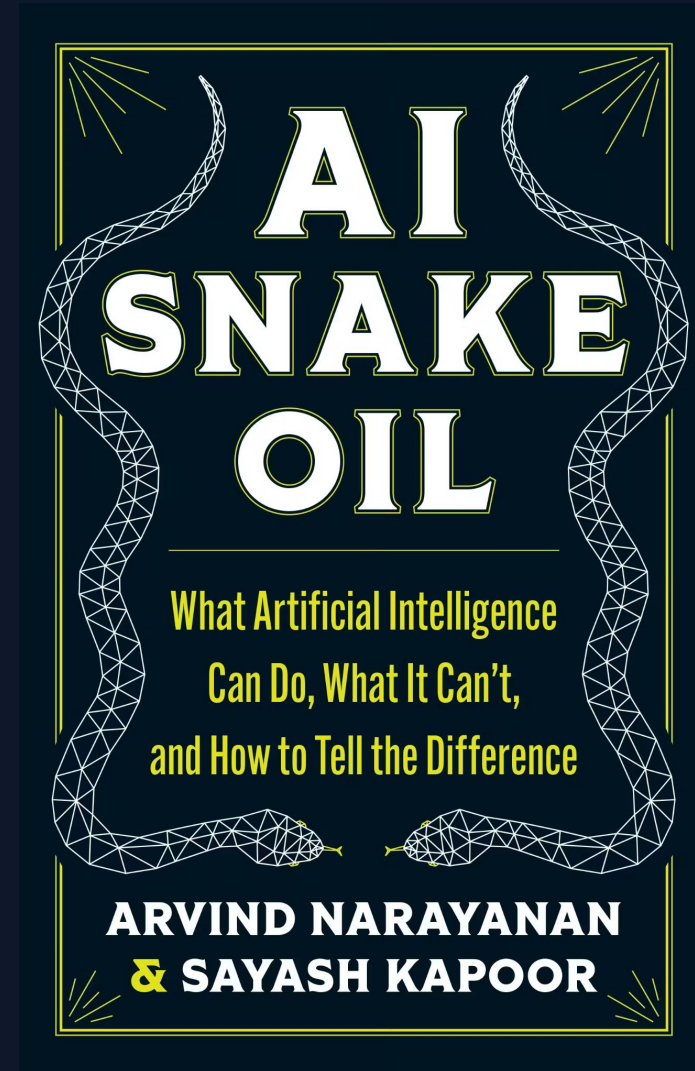
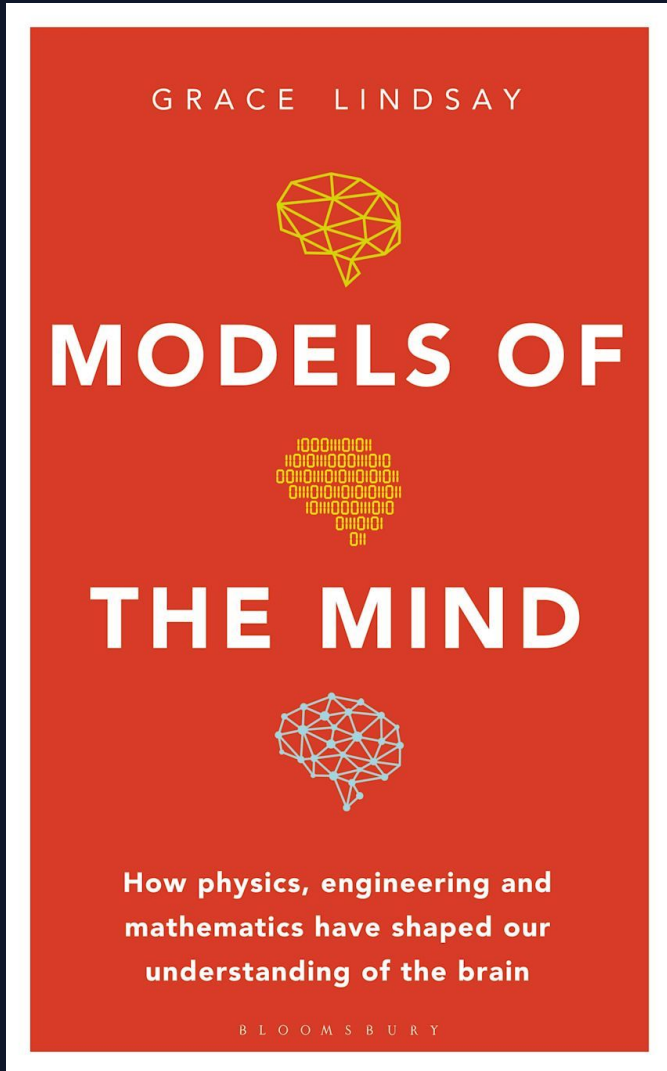
Decisions and analysis

- Take the best model: the one with the best precision in testing, of all possible models alongside the E training continuum.
- Add graphs to help visual examination.
- Analyse results and internal functioning.
- Deploy (as needed).

Literature



Literature recommendation



References

1. Russell, S. and Norvig, P. (2020). Artificial intelligence. A Modern Approach. Pearson.
2. Müller, A. and Guido, S. (2017). Introduction to Machine Learning with Python. A Guide for Data Scientists. O'Reilly.
3. Geron, A. (2019) Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. 2nd Edition, O'Reilly Media, Inc.
4. Koblah, D., Acharya, R., Capecci, D., Dizon-Paradis, O., Tajik, S., Ganji, F., Woodard, D., & Forte, D. (2023). A survey and perspective on artificial intelligence for security-aware electronic design automation. ACM Transactions on Design Automation of Electronic Systems, 28(2)

Lecturers note: This presentation lacks citations and may have typos. please do not share outside the course until this issues are adjusted.